

MODUL 12. Laravel : Database 1

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dan implementasi CodeIgniter pada pembuatan aplikasi berbasis *web*.
2. Mahasiswa mampu menerapkan CRUD menggunakan Laravel dan konsep MVC.
3. Mahasiswa mampu mengimplementasikan fitur-fitur yang sering digunakan pada Laravel.

12.1 CRUD

Pada modul ini kita akan membuat aplikasi yang dapat melakukan create, read, update, dan delete terhadap suatu data/tabel. Aplikasi yang akan kita bangun yaitu aplikasi e-commerce dimana kita akan fokus melakukan CRUD terhadap tabel produknya saja.

12.1.1 Konfigurasi dan Skema

Hal pertama yang harus dilakukan agar aplikasi dapat terhubung dengan database adalah mengatur konfigurasi koneksi. Secara detail, konfigurasi database berada pada file **config/database.php**. Tetapi untuk konfigurasi standar, kita dapat mengubahnya di file **.env** pada baris yang berisi **DB_CONNECTION** hingga **DB_PASSWORD**. Ubah nilainya dengan pengaturan yang kita inginkan.

```
...
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ecommerce
DB_USERNAME=root
DB_PASSWORD=
DB_CHARSET=utf8mb4
DB_COLLATION=utf8mb4_unicode_
ci
...
```

Buat database bernama **ecommerce** di DBMS. Untuk membuat tabelnya, dapat dilakukan secara otomatis (di-generate) oleh Laravel dari command prompt / console / terminal dengan perintah. Perintah yang pertama yaitu membuat file migrasi sebagai skema dari tabel:

```
php artisan make:migration create_products_table
```

File migrasi **xxx_create_products_table.php** akan terbuat pada folder **database/migrations**. Edit file ini pada fungsi **Schema::create()** untuk mendefinisikan kolom yang kita inginkan.

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->integer('price');
    $table->timestamps();
});
```

Penjabaran dari kode ini:

- Fungsi **id()** yaitu kita mendefinisikan kolom **id** sebagai PK dengan tipe int dan auto-increment.
- Fungsi **string('name')** yaitu kita mendefinisikan kolom **name** dengan tipe varchar
- Fungsi **integer('price')** yaitu kita mendefinisikan kolom **price** dengan tipe int
- Fungsi **timestamps()** yaitu kita mendefinisikan kolom **created_at** yang akan terisi jika data

dibuat melalui ORM dan **updated_at** yang akan terisi jika data diubah melalui ORM

Untuk membuat tabel ke database dari skema ini, maka ketikkan perintah berikut pada command prompt / console / terminal:

```
php artisan migrate
```

12.1.2 Model

Laravel memberikan tiga cara untuk mengakses ataupun manipulasi data ke database: query langsung, query builder, dan ORM. Query langsung dan query builder menggunakan library Illuminate (**Illuminate\Support\Facades\DB**) sedangkan ORM menggunakan Eloquent, yang merupakan kelas extend dari Illuminate. File Model diletakkan pada folder **app/Models**. Untuk membuat Model dalam Laravel, dapat dilakukan dengan manual dengan menambahkan namespace "App\Models" dan meng-extend kelas base Model dari Eloquent (**Illuminate\Database\Eloquent\Model**) ataupun di-generate oleh Laravel dari command prompt / console / terminal dengan perintah:

```
php artisan make:model Product
```

Jika sebelumnya belum membuat Controller atau file migration-nya, perintah generate Model ini dapat ditambahkan parameter sehingga bisa sekaligus me-generate file Controller-nya (-c), file migration-nya (-m), ataupun keduanya langsung (-cm).

Eloquent memiliki beberapa konvensi / aturan yang harus diperhatikan yaitu:

- Nama sebuah Model "X" secara otomatis merepresentasikan tabel database bernama "xs". Contoh, Model Product akan merepresentasikan tabel **products**. Jika tabel yang direpresentasikan berbeda, maka harus ditambahkan atribut **protected \$table = 'nama_tabel'**; pada kelas Model.
- Kolom PK pada tabel bernama "id". Jika kolom PK bukan "id", maka harus ditambahkan atribut **protected \$primaryKey = 'nama_kolom_PK'**; pada kelas Model.
- Kolom PK pada tabel di-set auto-increment. Jika kolom PK tidak auto-increment, maka harus ditambahkan atribut **public \$incrementing = false**; pada kelas Model.
- Kolom PK pada tabel bertipe integer. Jika kolom PK bukan integer, maka harus ditambahkan atribut **protected \$keyType = 'string'**; pada kelas Model.
- Ada kolom "created_at" dan "updated_at" pada tabel. Jika tidak ada, maka harus ditambahkan atribut **public \$timestamps = false**; pada kelas Model.

Beberapa aturan lain juga dapat diatur seperti date format, koneksi DB yang berbeda, dan nilai default untuk atribut tertentu.

12.1.3 Controller

Setelah Model siap, kita tinggal memanggilnya pada Controller. Berikut kode lengkapnya dalam ProductController:

```
<?php

namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index()
```

```

{
    $products = Product::all();
    return view('products.index', ['products' => $products]);
}

public function create()
{
    return view('products.form', [
        'title' => 'Tambah',
        'product' => new Product(),
        'route' => route('products.store'),
        'method' => 'POST',
    ]);
}

public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);

    Product::create($validated);
    return redirect()->route('products.index')->with('success', 'Produk berhasil
ditambahkan');
}

public function show(Product $product)
{
    return view('products.show', compact('product'));
}

public function edit(Product $product)
{
    return view('products.form', [
        'title' => 'Edit',
        'product' => $product,
        'route' => route('products.update', $product),
        'method' => 'PUT',
    ]);
}

public function update(Request $request, Product $product)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);

    $product->update($validated);
    return redirect()->route('products.index')->with('success', 'Produk berhasil
diperbarui');
}

public function destroy(Product $product)
{
    $product->delete();
    return redirect()->route('products.index')->with('success', 'Produk berhasil
dihapus');
}
}

```

12.1.4 View

Untuk tampilannya, buat file dengan nama **index.blade.php** yang diletakkan di folder **resources/views/product** (folder product dibuat dahulu agar rapi).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Daftar Produk</title>
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container">
    <div class="row justify-content-center mt-5">
        <div class="col-md-8">
            @if(session('success'))
                <div class="alert alert-success">{{ session('success') }}</div>
            @endif
            <div class="d-flex justify-content-between align-items-center mb-3">
                <span>{{ session('msg') }}</span>
                <a href="{{ route('products.create') }}" class="btn btn-
primary">Tambah</a>
            </div>
            <table class="table table-bordered table-striped">
                <thead>
                    <tr>
                        <th>Nama</th>
                        <th>Harga</th>
                        <th>Aksi</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach($products as $product)
                        <tr>
                            <td>{{ $product->name }}</td>
                            <td>{{ $product->price }}</td>
                            <td>
                                <a href="{{ route('products.edit', $product->id) }}"
class="btn btn-sm btn-primary">Edit</a>
                                <form method="POST" action="{{ route('products.destroy',
$product->id) }}" style="display:inline" onsubmit="return confirm('Yakin hapus?')">
                                    @csrf
                                    @method('DELETE')
                                    <button class="btn btn-sm btn-danger">Hapus</button>
                                </form>
                            </td>
                        </tr>
                    @endforeach
                </tbody>
            </table>
        </div>
    </div>
    <script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></
script>
</body>
</html>
```

Dan untuk tampilan form tambah dan edit, dapat dibuat satu halaman saja karena hampir sama. Hanya perlu tambahan pengkondisian di beberapa tempat.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form {{ $title }} Produk</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container">
  <div class="row justify-content-center mt-5">
    <div class="col-md-6">
      <h4>Form {{ $title }} Produk</h4>
      <form class="border p-4" method="POST" action="{{ $route }}">
        @csrf
        @if($method === 'PUT')
          @method('PUT')
        @endif
        <div class="mb-3">
          <label for="name" class="form-label">Nama</label>
          <input type="text" name="name" id="name" class="form-control
@error('name') is-invalid @enderror" value="{{ old('name', $product->name) }}">
          @error('name')
            <div class="invalid-feedback">{{ $message }}</div>
          @enderror
        </div>
        <div class="mb-3">
          <label for="price" class="form-label">Harga</label>
          <input type="number" name="price" id="price" class="form-
control @error('price') is-invalid @enderror" value="{{ old('price', $product-
>price) }}">
          @error('price')
            <div class="invalid-feedback">{{ $message }}</div>
          @enderror
        </div>
        <div class="text-center">
          <button type="submit" class="btn btn-
success">Simpan</button>
        </div>
      </form>
    </div>
  </div>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.j
s"></script>
</body>
</html>
```

12.1.5 Tampilan Halaman

Jalankan aplikasi. Jika *kode* yang dibuat sesuai dengan semua gambar di atas pada modul ini, maka tampilan aplikasi *webnya* akan seperti gambar dibawah ini.

1. <http://localhost:8000/products>



Gambar 12.1 Tampilan halaman view

2. <http://localhost:8000/products/create>

Form Tambah Produk

Nama

Harga

Gambar 12.2 Tampilan halaman form tambah produk

3. <http://localhost:8000/products>

The screenshot shows the same web interface as before, but the table now contains one row of data. The 'Aksi' column has two buttons: 'Edit' (blue) and 'Hapus' (red). A blue 'Tambah' button is still in the top right corner.

Nama	Harga	Aksi
Laptop	10.000.000	Edit Hapus

Gambar 12.3 Tampilan halaman view setelah tambah data

4. [http://localhost:8000/products/\[id\]/edit](http://localhost:8000/products/[id]/edit)

Form Edit Produk

Nama

Harga

Gambar 12.4 Tampilan halaman form ubah data

12.2 Templating Halaman

Sebelumnya kita sudah menggunakan beberapa directive Blade. Directive ini juga dapat digunakan untuk templating halaman, yaitu bagian-bagian dari halaman yang sama / berulang untuk semua halaman dapat dibuat menjadi satu file Blade, dan semua halaman itu dapat me-load-nya tanpa harus ditulis ulang.

Langkah pertama dalam templating adalah membuat file template-nya, yang dimiliki oleh tiap halaman. Untuk contoh kasus pada modul ini, kita membuat file template dengan nama **template.blade.php** dengan isi:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>@yield('title')</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <script src="{{ asset('js/jquery.min.js') }}"></script>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
"></script>
</head>
<body style="width:95%">
  @yield('content')
</body>
</html>
```

Directive **@yield** digunakan untuk menandakan bahwa di bagian itulah yang akan berbeda untuk tiap halamannya. Langkah selanjutnya adalah mengubah tampilan di tiap halaman seperti contoh **index.blade.php** di bawah ini:

```
@extends('template')

@section('title', 'Daftar Produk')

@section('content')
<body class="container">
  <div class="row justify-content-center mt-5">
    <div class="col-md-8">
      @if(session('success'))
        <div class="alert alert-success">{{ session('success') }}</div>
      @endif
      <div class="d-flex justify-content-between align-items-center mb-3">
        <span>{{ session('msg') }}</span>
        <a href="{{ route('products.create') }}" class="btn btn-
primary">Tambah</a>
      </div>
      <table class="table table-bordered table-striped">
        <thead>
```



```

        <tr>
            <th>Nama</th>
            <th>Harga</th>
            <th>Aksi</th>
        </tr>
    </thead>
    <tbody>
        @foreach($products as $product)
            <tr>
                <td>{{ $product->name }}</td>
                <td>{{ $product->price }}</td>
                <td>
                    <a href="{{ route('products.edit', $product->id) }}"
class="btn btn-sm btn-primary">Edit</a>
                    <form method="POST" action="{{ route('products.destroy',
$product->id) }}" style="display:inline" onsubmit="return confirm('Yakin
hapus?') ">
                        @csrf
                        @method('DELETE')
                        <button class="btn btn-sm btn-danger">Hapus</button>
                    </form>
                </td>
            </tr>
        @endforeach
    </tbody>
</table>
</div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js
"></script>
</body>
@endsection

```

Directive **@extend** digunakan untuk menentukan file template mana yang digunakan oleh halaman ini. Sedangkan directive **@section** digunakan untuk mengisi halaman sesuai dengan nama **yield** yang di-define dalam file template. Directive **@section** dapat diisi dengan string ataupun tag HTML.

12.3 Form Validation

Laravel dapat ditambahkan form validation dari sisi server. Di controller, sebelum kita memanggil fungsi untuk menambahkan (fungsi **store**) / mengubah (fungsi **update**), kita dapat menambahkan kode:

```

...
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);
    ...
public function update(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);
    ...

```

Di kode validasi ini, kita mengatur nama tidak boleh kosong dan minimal 4 karakter, sedangkan harga tidak boleh kosong, harus integer, dan nilai minimal 1.000.000. Kemudian di halaman **form.blade.php** kita harus menambahkan directive **@error** seperti ini:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Form {{ $title }} Produk</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container">
    <div class="row justify-content-center mt-5">
        <div class="col-md-6">
            <h4>Form {{ $title }} Produk</h4>
            <form class="border p-4" method="POST" action="{{ $route }}">
                @csrf
                @if($method === 'PUT')
                    @method('PUT')
                @endif
                <div class="mb-3">
                    <label for="name" class="form-label">Nama</label>
                    <input type="text" name="name" id="name" class="form-control
@error('name') is-invalid @enderror" value="{{ old('name', $product->name) }}">
                    @error('name')
                        <div class="invalid-feedback">{{ $message }}</div>
                    @enderror
                </div>
                <div class="mb-3">
                    <label for="price" class="form-label">Harga</label>
                    <input type="number" name="price" id="price" class="form-
control @error('price') is-invalid @enderror" value="{{ old('price', $product-
>price) }}">
                    @error('price')

```

```

        <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>
    <div class="text-center">
        <button type="submit" class="btn btn-success">Simpan</button>
    </div>
</form>
</div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
></script>
</body>
</html>

```

Directive **@error** diletakkan di atribut class pada tag <input> untuk validasinya dan diletakkan di bawah tag <input> untuk menampilkan pesan validasinya. Kemudian agar data pada form di-repopulate, maka isikan atribut value pada tag <input> dengan **old('value-di-atribut-name')**.

MODUL 13. Laravel : Database 2

Tujuan Praktikum

4. Mahasiswa mampu memahami konsep dan implementasi CodeIgniter pada pembuatan aplikasi berbasis *web*.
5. Mahasiswa mampu menerapkan CRUD menggunakan Laravel dan konsep MVC.
6. Mahasiswa mampu mengimplementasikan fitur-fitur yang sering digunakan pada Laravel.

13.1 Session

Session memiliki fungsi menyimpan suatu variabel pada server dan variabel ini dapat diakses dan diubah dimanapun: Routing, Controller, ataupun halaman manapun. Session default pada Laravel dikelola sendiri oleh Laravel dan disimpan dalam bentuk file. Selain file, Session pada Laravel juga dapat diubah menjadi disimpan ke database atau menggunakan mekanisme lain.

Laravel memiliki dua jenis Session, yaitu Session biasa dan Session flash. Session biasa akan selalu ada selama belum dihapus, time-out, atau browser ditutup. Sedangkan Session flash adalah session yang hanya berlaku untuk satu request saja, setelah itu Laravel akan menghapusnya secara otomatis. Contoh Session flash sudah pernah kita gunakan melalui fungsi **with('x', y)** yang digunakan bersamaan dengan fungsi **redirect**. Untuk Session biasa, berikut contoh penggunaannya di Routing:

```
...
Route::get('/login', function () {
    if (session()->has('email')) return redirect('/product');
    return view('login');
});

Route::get('/logout', function () {
    session()->flush();
    return redirect('/login');
});
...
```

Fungsi **session()->has('x')** digunakan untuk memeriksa apakah ada Session bernama "x". Sedangkan **session()->flush()** digunakan untuk menghapus semua Session. Contoh lain, berikut penggunaan Session di SiteController:

```
...
public function auth(Request $req) {
    $u = User::where([
        ['email', $req->em],
        ['password', $req->pwd],
    ])->first();
    if (isset($u)) {
```

```

    session()->put('email', $u->email);
    session()->put('name', $u->name);
    return "<script>
    alert('Welcome, " . session('name') . "');
    location.href='/product';
    </script>";
}
return redirect('/login')->with('msg', 'Email / password salah');
}
...

```

Fungsi **session()->put('x', y)** digunakan untuk membuat Session bernama "x" dengan nilai y. Sedangkan **session('x')** digunakan untuk mengambil nilai dari Session "x".

13.2 Middleware

Middleware pada Laravel adalah suatu mekanisme untuk menyaring request yang masuk ke suatu Routing. Sebagai contoh, kita bisa membuat Middleware untuk memverifikasi apakah seorang user sudah terautentikasi atau memiliki hak akses untuk mengakses URL. Jika user belum terautentikasi atau tidak memiliki akses, maka Middleware dapat me-redirect user tersebut ke URL lain. Sebaliknya jika user terautentikasi atau memiliki hak akses, maka Middleware akan meneruskan request ke aksi yang dipetakan di Routing.

Middleware dapat melakukan hal lain selain untuk autentikasi, misalnya untuk menulis log atau error yang terjadi. Dan Middleware dapat kita atur mekanisme yang dilakukannya, yaitu bisa sebelum request diteruskan atau sesudah request diteruskan. Middleware yang kita buat harus berada di folder **app/Http/Middleware**.

Sekarang kita akan membuat autentikasi dengan menggunakan library **Auth**. Library ini sudah termasuk di dalam paket instalasi Laravel, sehingga kita tidak perlu menambahkannya lagi melalui composer. Dalam library ini sudah mencakup semua fitur untuk autentikasi, registrasi, dan middleware-nya. Dengan menggunakan **Auth**, autentikasi menjadi lebih simpel, aman, dan bisa menjadi alternatif pengganti yang lebih baik dari Session.

Langkah pertama adalah pengaturan pada Routing. Ganti Routing untuk login, logout, dan product:

```

...
Route::get('/login', function () {
    if (Auth::check()) return redirect('/product');
    return view('login');
})->name('login');

Route::get('/logout', function () {
    Auth::logout();
    return redirect('/login');
});

Route::resource('product', ProductController::class)->middleware('auth');

```

Fungsi **Auth::check()** digunakan untuk memeriksa apakah user telah terautentikasi. Routing login kita berikan nama alias karena kebutuhan dari Middleware **auth**. Sedangkan **Auth::logout()** digunakan untuk menghapus autentikasi. Untuk product, Middleware kita ganti dengan Middleware **auth** yang berkolaborasi dengan library Auth.

Lalu langkah kedua ubah isi dari fungsi **auth** di **SiteController**:

```

...
use Illuminate\Support\Facades\Auth;

class SiteController extends Controller
{
    public function auth(Request $req) {
        if (Auth::attempt(['email'=>$req->em, 'password'=>$req->pwd])) {
            //jika ada nilai lain selain data user yang ingin disimpan di session, baru
            gunakan session disini
            return redirect('/product');
        }
        return redirect('/login')->with('msg', 'Email / password salah');
    }
}
...

```

Fungsi **Auth::attempt()** digunakan untuk proses autentikasi, yaitu apakah email dan password yang di-submit ada dalam database pada tabel **users**. Jika email dan password cocok maka akan menghasilkan **true**. Hal yang harus diperhatikan adalah ketika menambahkan data User ke database, pastikan isi dari kolom password di-hash dengan metode **Bcrypt** agar dapat menggunakan library Auth ini. Laravel sudah menyediakan fungsi **bcrypt('x')** untuk mempermudahnya.

Langkah ketiga, dalam View yang sebelumnya menggunakan **@if(session('x'))** dapat kita ganti menjadi directive **@auth** untuk pengecekan apakah user telah terautentikasi, dan dapat menggunakan **Auth::user()** untuk mengakses data user yang login. Berikut contohnya dalam **template.blade.php**:

```

...
<body style="width:95%">
    @auth
    <div class="row justify-content-end" style="margin-top:2%">
        <div class="col-3">
            {{ Auth::user()->name }}
            <a href="/logout" class="btn btn-warning">Logout</a>
        </div>
    </div>
    @endauth
    <div class="row justify-content-center" style="margin-top:10%">
        @yield('content')
    </div>
...

```

13.3 Model Relasi

Model Eloquent menyediakan fasilitas agar dua Model yang berelasi dapat langsung memanggil satu sama lain. Kita akan mencoba salah satunya yaitu relasi one to many, dengan menggunakan contoh kasus relasi **Product** dengan **Variant**. Tiap **Product** dapat memiliki banyak **Variant**, tetapi tiap **Variant** hanya dimiliki oleh satu **Product**. Langkah awal yaitu membuat Model dan file migration-nya, maka perintahnya:

```
php artisan make:model Variant -m
```

Setelah itu edit file **xxx_create_variants_table.php** pada folder **database/migrations** untuk mendefinisikan kolomnya:

```

...
Schema::create('variants', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->text('description');
    $table->string('processor');
    $table->string('memory');
    $table->string('storage');
    $table->foreignId('product_id')->constrained();
    $table->timestamps();
});
...

```

Untuk kebutuhan foreign key, kita menggunakan **foreignId** dan **constrained** jika tabel entitas yang diacu mengikuti konvensi Model Eloquent (PK bernama “id”, tabel database ditambah akhiran “s”, dsb). Jika berbeda, maka pendefinisian foreign key harus menggunakan **foreign**, **references**, dan **on**. Contoh:

```

...
$table->integer('product_id');
$table->foreign('product_id')->references('id_product')->on('product');
...

```

Kemudian generate tabel **variants** dengan perintah seperti sebelumnya (php artisan migrate). Selanjutnya edit file Model **Variants**, tambahkan fungsi berikut:

```

...
public function product() {
    return $this->belongsTo(Product::class);
}
...

```

Fungsi **belongsTo()** secara otomatis akan memanggil object **Product** yang berelasi dengan dirinya berdasarkan **product_id**. Kemudian tambahkan juga fungsi ke dalam Model **Product**:

```

...
public function variants() {
    return $this->hasMany(Variant::class);
}
...

```

Fungsi **hasMany()** secara otomatis akan memanggil semua object **Variant** yang berelasi dengan dirinya berdasarkan **product_id**. Untuk mencobanya kita akan menambahkan satu kolom pada tampilan tabel di **index.blade.php** untuk menampilkan data variant dari tiap product:

```

...
<th>Harga</th>
<th>Variant</th>
<t

    h>Aksi</th>
</tr>
@foreach($list as $d)
<tr>
    <td>{{ $d->name }}</td>
    <td>{{ $d->price }}</td>

```

```

<td>
<ul>
@foreach($d->variants()->get() as $var)
    <li>{{ $var->name }}</li>
    Desc: {{ $var->description }} <br /> Proc:
    {{ $var->processor }} <br />
    RAM: {{ $var->memory }} <br /> Strg:
    {{ $var->storage }} <br /> Product: {{
    $var->product->name }}
    @endforeach
</ul>
</td>
<td align="center">

```

...

Tiap data product dapat langsung mengakses semua data variant-nya dengan fungsi variants() dan tiap variant dapat mengakses product yang berelasi dengannya dengan menggunakan atribut product. Uji dengan memasukkan data variants di database dan membuat form untuk menambahkan data variants ke database.